1 Choose the right script

Each script contains a Python function and a dictionary that tells Workbench how to display and call it.

Processing: create an object in NWB

Input: a selected NWB object and parameters, once per file.
Output: a new NWB object returned to the runner.
Declaration: TOOL, normally created with make_file_tool.

Analysis: calculate and plot results

Input: prepared data grouped by condition, plus parameters.
Output: figures and tables registered with result.add.
Declaration: ANALYSIS.

Spike preprocessing: transform a recording

Input: a SpikeInterface recording.
Output: a SpikeInterface recording returned to the sorting workflow.
Declaration: PREPROCESSING.

Save project-specific files here

Processing: **Script/Processing/Examples/**
Analysis: **Script/Analysis/Examples/**
Spike preprocessing: **Script/Spike_sorting/Preprocessing/**

Paths are relative to your Workbench project. Shared built-ins are in Processings/, Analyses/ and Spike_sorting/Preprocessing/ inside the main package.

Save a .py file, restart Workbench and look for its declared name. Use a unique name. Keep code that reads data or runs calculations inside the function, not at module import.

2 Processing: the function

Example: subtract the temporal mean from each channel while retaining the original timing.

Save this and the next page together as **Script/Processing/Examples/demean_signal.py**.

```
import numpy as np
from pynwb import TimeSeries
from GUI.processing_runner import make_file_tool


def demean_signal(data, saving_name, **kwargs):
    if data is None:
        raise ValueError("Select an input signal.")
    if not saving_name or "/" in saving_name:
        raise ValueError("Enter a valid output name.")
    values = np.asarray(data.data[:], dtype=float)
    if values.ndim not in (1, 2) or values.shape[0] == 0:
        raise ValueError("Expected samples or samples-by-channels.")
    if getattr(data, "channel_conversion", None) is not None:
        raise ValueError("Channel conversion is not supported.")
    values = values * float(data.conversion) + float(data.offset)
    values -= np.nanmean(values, axis=0, keepdims=True)
    if data.timestamps is not None:
        timing = {"timestamps": np.asarray(data.timestamps[:])}
    elif data.rate is not None:
        timing = {"rate": float(data.rate),
                  "starting_time": float(data.starting_time)}
    else:
        raise ValueError("No usable timing information.")
    print(f"Demeaned {data.name}: {values.shape}", flush=True)
    return TimeSeries(
        name=saving_name, data=values, unit=data.unit,
        description="Temporal mean removed from " + data.name,
        **timing,
    )
```

Input data is a PyNWB object here. data.data[:] loads its stored values. conversion and offset map them to physical units. The returned TimeSeries uses those units and preserves timestamps or rate.

The example returns a generic TimeSeries and rejects channel-specific conversion. It does not preserve an ElectricalSeries electrode association. Adapt this deliberately for your data.

3 Processing: the TOOL

Append this declaration below the function. The helper supplies Workbench's per-file entry point.

```
TOOL = make_file_tool(
    name="Example: demean signal",
    description="Remove each channel's temporal mean.",
    process_one=demean_signal,
    continue_on_error=False,
    debug_mode=False,
    params={
        "category": "Examples",
        "data": {
            "type": "data", "label": "Input signal",
            "required": True,
            "neurodata_type_in": ["TimeSeries", "ElectricalSeries"],
            "load_data": False,
        },
        "saving_name": {
            "type": "saving_name", "label": "Output name",
            "default": "demeaned_signal", "required": True,
        },
    },
)
```



Real GUI for this example: 1 input; 2 output name; 3 Run.

load_data=False passes the NWB object. True passes only its stored array. **saving_name** supplies the output name. **params.category** selects the output processing module. Parameter keys match function arguments.

Select one test file, choose the tool, then Run. The output appears under Content > Processing > Examples. Return the object; do not also attach it manually. A raw TOOL func receives a transfer object, so use the helper for this pattern.

4 Analysis: the function

Example: one figure per input, with a pooled mean curve for each condition.

Save this and the next page together as **Script/Analysis/Examples/mean_trace.py**.

```
import pandas as pd
import matplotlib.pyplot as plt


def mean_trace(data_manager, result, title="Mean trace", **kwargs):
    for key in data_manager.get_data_types():
        fig, ax = plt.subplots()
        tables = []
        for condition in data_manager.get_conditions():
            data = data_manager.get_data(key, condition)
            if "time" not in getattr(data, "dims", ()):
                raise ValueError("Expected an xarray time dimension.")
            dims = [d for d in data.dims if d != "time"]
            trace = data.mean(dim=dims, skipna=True) if dims else data
            if trace.size == 0:
                raise ValueError("No samples in this condition.")
            ax.plot(trace.time.values, trace.values,
                    label=str(condition))
            tables.append(pd.DataFrame({
                "condition": str(condition),
                "time_s": trace.time.values,
                "mean_value": trace.values,
            }))
        if not tables:
            plt.close(fig)
            raise ValueError("No conditions were provided.")
        ax.set(title=f"{title}: {key}", xlabel="Time (s)",
               ylabel="Mean signal (input units)")
        ax.legend()
        fig.tight_layout()
        result.add(fig, data=pd.concat(tables, ignore_index=True))
        print(f"Created mean trace: {key}", flush=True)
```

get_data returns prepared data, often an xarray object. This example requires a time dimension and averages all other dimensions. It is a teaching example, not a subject-level statistical estimator.

result.add(fig, data=table) registers both outputs for display and saving. Returning a table alone does not register it. print(..., flush=True) makes progress visible in the analysis log.

5 Analysis: the ANALYSIS dictionary

Append this declaration. It defines the input selector, event alignment and scientific parameters.

```
ANALYSIS = {
    "name": "Example: mean trace",
    "description": "Pooled mean over all dimensions except time.",
    "func": mean_trace,
    "align_to_event_marker": True,
    "debug_mode": False,
    "params": {
        "title": {
            "type": "str", "label": "Figure title",
            "default": "Mean trace",
        },
    },
    "input_spec": {
        "accepted_data": ["TimeSeries", "ElectricalSeries"],
        "allow_multiple": True,
        "required": True,
    },
}
```



1 Input data; 2 Start/End alignment; 3 title parameter. Controls come from the declaration.

accepted_data limits offered NWB types. **allow_multiple=True** allows Add data. **required=True** requires a selection. **align_to_event_marker=True** asks Workbench to prepare aligned windows.

Create conditions, choose data and alignment, then Run. Tick Save analysis to retain figures, associated tables and the log. The standard method receives prepared data through data_manager, not an open NWB object.

6 Spike preprocessing: script

Example: apply a bandpass filter to the recording before sorting.

Save as **Script/Spike_sorting/Preprocessing/project_bandpass.py**.

```
import spikeinterface.preprocessing as spre


def bandpass(recording, freq_min=300.0, freq_max=6000.0):
    nyquist = recording.get_sampling_frequency() / 2
    if not 0 < freq_min < freq_max < nyquist:
        raise ValueError("Require 0 < low < high < Nyquist.")
    return spre.bandpass_filter(
        recording, freq_min=freq_min, freq_max=freq_max
    )


PREPROCESSING = {
    "name": "Project bandpass",
    "description": "Bandpass before spike sorting.",
    "func": bandpass,
    "default": False,
    "params": {
        "freq_min": {"type": "float", "default": 300.0},
        "freq_max": {"type": "float", "default": 6000.0},
    },
}
```

recording is a SpikeInterface recording. Read its sampling rate with get_sampling_frequency() and sample blocks with get_traces(). The function must return a recording, not a NumPy array or NWB TimeSeries.

The transform may be lazy: samples are filtered when requested. Test get_traces() as well as creating the object. The example frequencies require a sampling rate above 12 kHz and must suit your experiment.

default=False leaves the operation disabled initially. **params** creates its editable settings. This operation belongs to Spike sorting > Preprocessing, not the main Processing tab.

7 Enable and check preprocessing

After saving the script, restart Workbench and open the sorting project's Preprocessing options.



Real options dialog with the example operation. 1 enable; 2 order; 3 parameter values.



Apply preprocessing commits the configured operations to this sorting workflow.

1. Enable **Project bandpass** (1). Set its order (2) relative to other enabled operations.

2. Check the frequency values (3), then click **Apply preprocessing** (4).

3. Inspect representative traces and channel identities before running the sorter.

4. Keep the source recording available: a recording transform can still depend on it.

Input and output must remain compatible

Preserve sampling frequency, channel identities, sample counts and segment structure unless your operation explicitly handles a change. Do not flatten a multi-segment recording into an array and lose its timing.

This chain changes the sorter's input. It does not automatically create a derived object in the source NWB Processing branch.

8 Test before a batch

A script appearing in the menu proves that it imported, not that its result is correct.

Four checks for every new script

1. Test known numbers. Use a small synthetic input whose expected result you can calculate.

2. Check the contract. Verify output type, shape, timing, units and channel identity as applicable.

3. Run one real example. Use a copied NWB file when testing a tool that writes data. Inspect the result in the GUI.

4. Reopen saved outputs. Confirm the figure/table, derived NWB object or transformed recording contains the expected values.

Useful parameter fields

label: visible caption. **default:** initial value. **required:** mandatory value. **type:** str, int, float or bool. For Processing/Analysis choices, use type="choice" with choices, or type="list" with values.

Processing/Analysis numeric **min/max** constrain form values. Always validate scientific limits inside the function too. Parameter names become function argument names; **kwargs accepts extra runner context.

When something goes wrong

Not in the menu: check the folder, .py extension, declaration and import errors in project Logs. Dependencies must be installed in the execution environment.

Wrong/empty data: inspect dimensions, coordinates and retained trials. Aligned segments can contain NaN padding. Spike times are not necessarily regularly sampled traces.

Nothing saved: Processing must return a supported object. Analysis must register a figure and associated data with result.add. Spike preprocessing must return a recording.

For advanced fields, raw-NWB analyses or custom runners, use the complete guide. The three examples here cover the normal starting routes.